SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO4 – Precision	**Assessment:**	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	35% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	VIMALA K	**Reg. No.:**	192424276
Section / Batch:	B.Tech(AI&DS)	**Date:**	20/08/2026

Code of Conduct

*I **VIMALA K 192424276** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: 

Instructions

- Read each industry scenario carefully before answering.
- Analyze the given problem from a semantic analysis perspective.
- Provide structured and logical answers with proper justification.
- Support your analysis using examples from the given scenario wherever applicable.
- Use suitable diagrams, semantic representations, logical predicates, workflows, architectures, or tables whenever necessary.
- Clearly state assumptions if additional information is required.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze semantic interpretation of user queries, identify ambiguity in meaning representation, evaluate semantic processing limitations, and improve understanding in banking chatbot applications.	Q1
Syntax-Driven Semantic Analysis	Analyze multiple interpretations of spoken commands, evaluate parsing-related ambiguity, and assess semantic extraction in real-time voice assistant systems.	Q2
First-Order Predicate Calculus, Representing Linguistically Relevant Concepts, Syntax-Driven Semantic Analysis	Design a semantic processing architecture for healthcare reports, model relationships among entities and actions, represent linguistic concepts, and improve semantic extraction accuracy.	Q3

Annexure A – Industry Problem Solving Task

1. AI-Powered Insurance Claim Processing System

An insurance company is developing an NLP-based system to automatically process customer insurance claims. The system uses **Context-Free Grammar (CFG)** to parse customer statements and extract information such as the claimant, damaged object, cause of damage, and claim amount.

However, the system faces the following problems:

- Ambiguous sentence structures
- Difficulty distinguishing noun and prepositional phrase attachment
- Subject–verb agreement errors
- Inefficient parsing of lengthy customer descriptions
- Difficulty handling conversational and incomplete input

Example customer statement:

"I reported the damage to the car after the accident."

The sentence can produce different interpretations depending on whether **"to the car"** is associated with *reported* or *damage*.

Tasks:

a) Analyze the structural ambiguity in the given sentence using CFG and illustrate the possible interpretations.

b) Evaluate the limitations of a basic CFG in handling ambiguity, agreement, and long customer-generated insurance statements.

c) Design an improved parsing approach using **PCFG, Feature Structures, and Earley Parsing** to resolve ambiguity and efficiently process the claim.

d) Justify how the proposed architecture can improve the accuracy, scalability, and reliability of automated insurance claim processing.

```
print("=" * 65)
print("1. AI-POWERED INSURANCE CLAIM PROCESSING SYSTEM")
print("=" * 65)

sentence = "I reported the damage to the car after the accident."

print("\nTASK A: STRUCTURAL AMBIGUITY")
print("\nSentence:")
print(sentence)

print("\nInterpretation 1:")
print("I reported [the damage to the car] [after the accident].")
print("Meaning: 'to the car' describes the damage.")

print("\nInterpretation 2:")
print("I reported [the damage] [to the car] [after the accident].")
print("Meaning: 'to the car' describes where the report was made or directed.")

print("\nCFG Representation:")
print("S -> NP VP")
print("VP -> V NP PP")
print("NP -> Det N PP")
print("PP -> P NP")
print("NP -> Det N")
print("PP -> P NP")
```

```
print("\nTASK B: LIMITATIONS OF BASIC CFG")
print("""
1. CFG can generate multiple parse trees for ambiguous sentences.
2. Basic CFG does not assign probabilities to different parses.
3. It does not naturally handle subject-verb agreement.
4. It is inefficient for long and deeply nested sentences.
5. It has difficulty with incomplete conversational input.
6. It does not directly represent semantic information.
7. It may produce many possible parses for customer statements.
""")

print("TASK C: IMPROVED PARSING APPROACH")
print("""
PCFG:
Assigns probabilities to different parse trees and selects
the most probable interpretation.

Feature Structures:
Represent grammatical features such as:
- Number
- Person
- Tense
- Agreement

Earley Parsing:
Efficiently handles:
- Ambiguous grammar
- Incomplete input
- Left recursion
- Long sentences

Architecture:

Customer Statement
        |
        v
Text Preprocessing
        |
        v
CFG Grammar
        |
        v
Feature Structures
        |
        v
Earley Parser
        |
        v
PCFG Ranking
        |
        v
Semantic Role Extraction
        |
```

v
Insurance Claim Information
""")

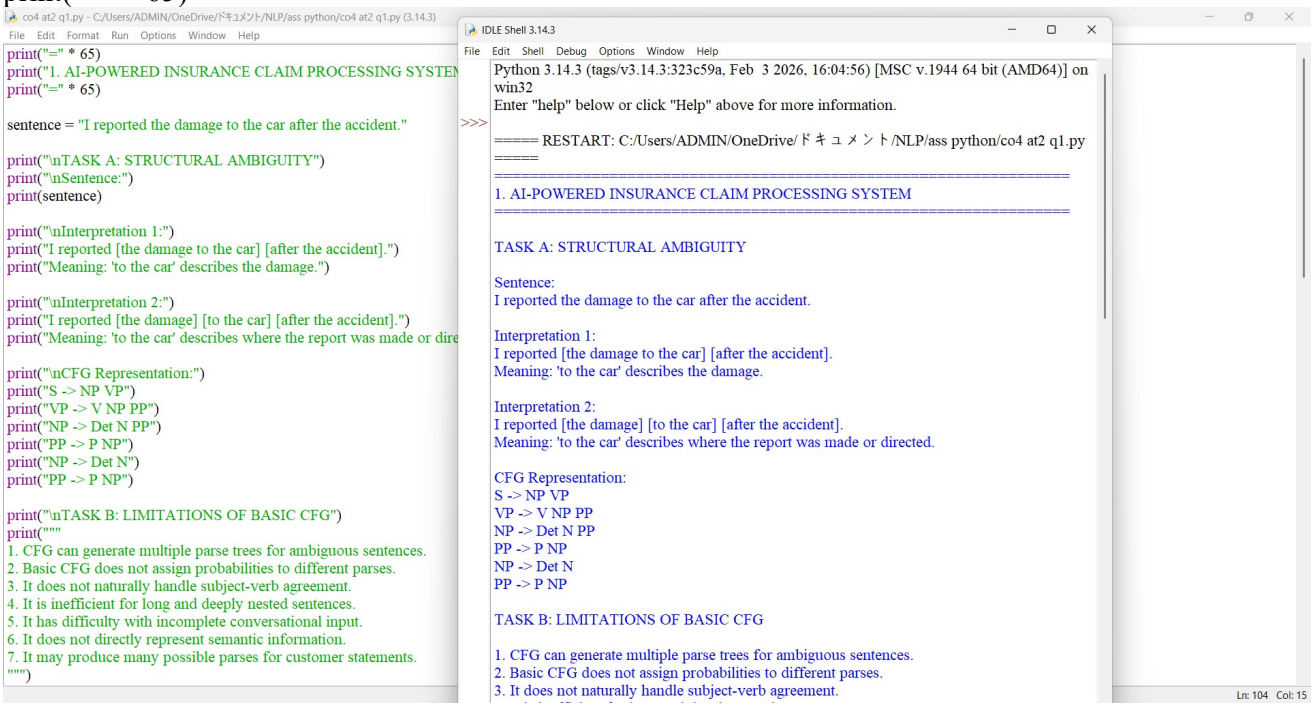
print("TASK D: BENEFITS")
print("""
Accuracy:
PCFG selects the most probable interpretation.

Scalability:
Earley parsing can handle complex grammar and long sentences.

Reliability:
Feature structures enforce agreement constraints.

The system can extract:
Claimant
Damaged Object
Cause of Damage
Claim Amount
""")

print("=" * 65)
print("INSURANCE CLAIM ANALYSIS COMPLETED")
print("=" * 65)



2. Intelligent Railway Ticket Booking Assistant
A railway company is developing a conversational AI assistant for ticket booking. The system currently uses **top-down parsing** to interpret passenger requests.
The system encounters:

- Excessive backtracking
- Ambiguous attachment of phrases

- Incomplete passenger requests
- Long-distance dependencies
- Slow response for complex queries

Example passenger request:
"Book two tickets from Chennai to Delhi for my parents with AC sleeper."
The system sometimes incorrectly associates **"with AC sleeper"** with the passengers instead of the ticket/class information.
Tasks:
a) Analyze the possible syntactic interpretations of the given passenger request using CFG.
b) Evaluate why top-down parsing may result in excessive backtracking and inefficient processing for conversational ticket-booking queries.
c) Analyze how **Earley Parsing** can handle ambiguity, incomplete input, and different possible parse structures more effectively.
d) Compare **Top-Down Parsing, CFG-based parsing, and Earley Parsing** in terms of ambiguity handling, computational efficiency, partial-input processing, and suitability for real-time railway booking systems.

```
print("=" * 65)
print("2. INTELLIGENT RAILWAY TICKET BOOKING ASSISTANT")
print("=" * 65)

sentence = "Book two tickets from Chennai to Delhi for my parents with AC sleeper."

print("\nTASK A: CFG INTERPRETATIONS")
print("\nPassenger Request:")
print(sentence)

print("\nInterpretation 1:")
print("[Book two tickets] [from Chennai] [to Delhi] [for my parents] [with AC sleeper].")
print("Meaning: AC sleeper is the ticket/class information.")

print("\nInterpretation 2:")
print("[Book two tickets] [from Chennai] [to Delhi] [for my parents with AC sleeper].")
print("Meaning: AC sleeper is incorrectly attached to the passengers.")

print("\nCFG Representation:")
print("S -> VP")
print("VP -> V NP PP PP PP PP")
print("NP -> Num N")
print("PP -> P NP")
print("NP -> Poss N")
print("NP -> Adj N")
print("PP -> P NP")

print("\nTASK B: TOP-DOWN PARSING LIMITATIONS")
print("""
1. Top-down parsing starts from the start symbol.
2. It predicts grammar rules before seeing all input.
3. Wrong predictions may require backtracking.
```

4. Ambiguous PP attachment can cause many alternatives.
5. Long passenger requests increase search effort.
6. Incomplete conversational requests are difficult to complete.
7. Complex queries can increase response time.
""")

print("TASK C: EARLEY PARSING")
print("""
Earley Parsing is useful because it can:

1. Handle ambiguous grammars.
2. Process incomplete input.
3. Avoid unnecessary repeated parsing.
4. Handle different possible parse structures.
5. Support long-distance dependencies.
6. Work with context-free grammars.

Example:

Book two tickets
    |
    v
From Chennai
    |
    v
To Delhi
    |
    v
For my parents
    |
    v
With AC sleeper
    |
    v
Ticket Class = AC Sleeper
""")

print("TASK D: COMPARISON")

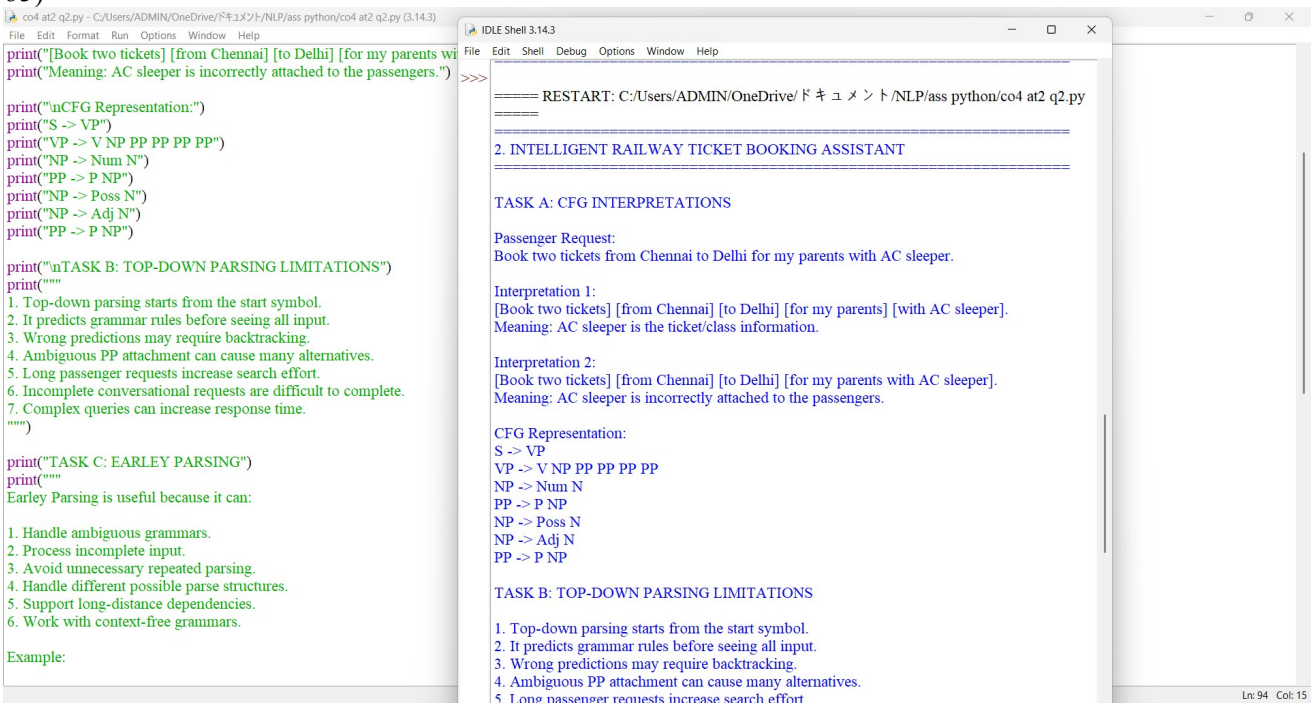
print("""
Top-Down Parsing:
- Ambiguity: Limited
- Backtracking: High
- Partial Input: Weak
- Real-Time Use: Less suitable for complex queries

CFG-Based Parsing:
- Ambiguity: Can generate multiple parses
- Backtracking: Depends on parser
- Partial Input: Limited
- Real-Time Use: Suitable for simple queries

Earley Parsing:

- Ambiguity: Handles multiple parses
- Backtracking: Reduced
- Partial Input: Good
- Real-Time Use: Suitable for complex conversational queries
""")

print("=" * 65)
print("RAILWAY BOOKING ANALYSIS COMPLETED")
print("=" * 65)



3. AI-Based Legal Contract Analysis System

A legal technology company is developing an NLP system to analyze contracts and automatically identify:

- Parties involved
- Obligations
- Conditions
- Deadlines
- Actions
- Exceptions

The existing system uses a basic CFG parser. However, it produces incorrect interpretations when processing long legal sentences containing relative clauses, passive constructions, and multiple prepositional phrases.

Example contract sentence:

"The supplier who delivered the equipment to the company must replace the defective components within thirty days."
The system sometimes incorrectly identifies the **company** as the entity responsible for delivering the equipment.
Tasks:
a) Analyze the syntactic structure of the given sentence using CFG and identify possible sources of ambiguity.
b) Evaluate why basic CFG is insufficient for accurately interpreting complex legal sentences containing relative clauses and nested structures.
c) Design an NLP architecture integrating **CFG, PCFG, Feature Structures, and Earley Parsing** to improve syntactic and semantic interpretation.
d) Develop a step-by-step processing workflow showing how the system can extract:

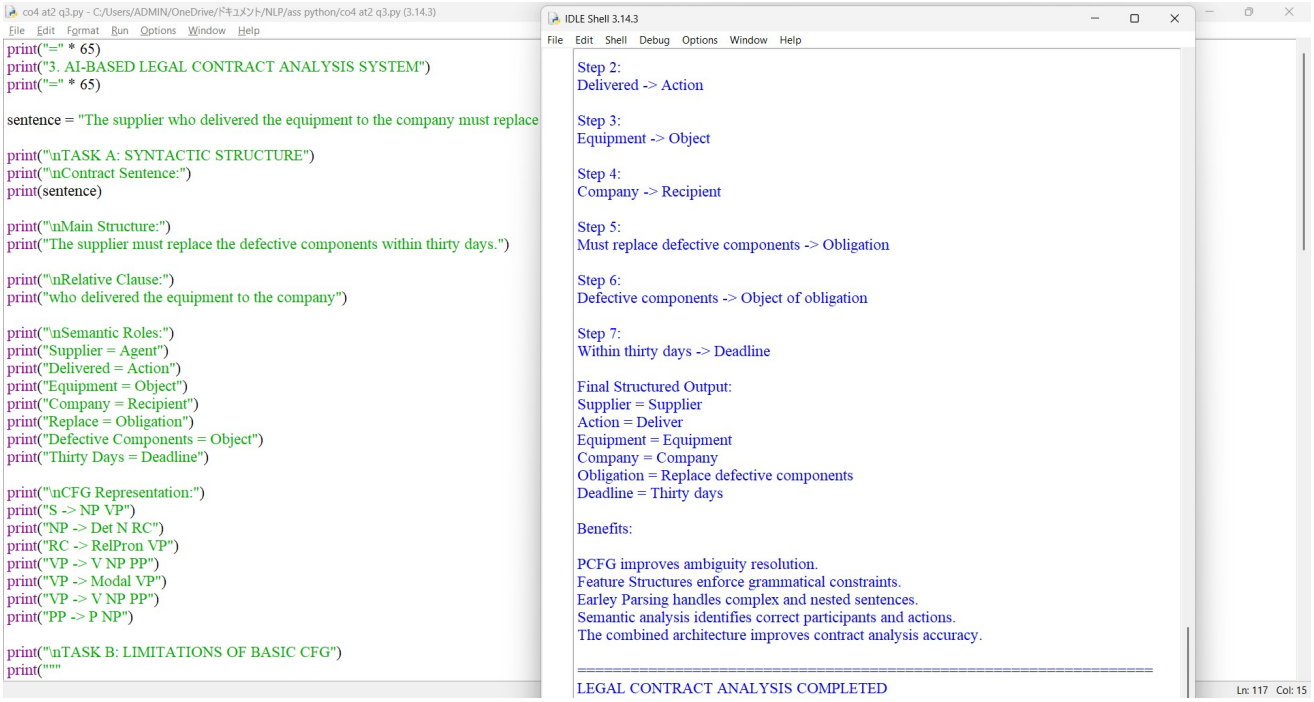
- Supplier
- Action
- Equipment
- Company
- Obligation
- Deadline

and justify how the proposed approach improves automated contract analysis.

```
print("=" * 65)
print("3. AI-BASED LEGAL CONTRACT ANALYSIS SYSTEM")
print("=" * 65)

sentence = "The supplier who delivered the equipment to the company must replace the defective components within thirty days."

print("\nTASK A: SYNTACTIC STRUCTURE")
print("\nContract Sentence:")
print(sentence)

print("\nMain Structure:")
print("The supplier must replace the defective components within thirty days.")

print("\nRelative Clause:")
print("who delivered the equipment to the company")

print("\nSemantic Roles:")
print("Supplier = Agent")
print("Delivered = Action")
print("Equipment = Object")
print("Company = Recipient")
print("Replace = Obligation")
print("Defective Components = Object")
print("Thirty Days = Deadline")

print("\nCFG Representation:")
print("S -> NP VP")
```

```
print("NP -> Det N RC")
print("RC -> RelPron VP")
print("VP -> V NP PP")
print("VP -> Modal VP")
print("VP -> V NP PP")
print("PP -> P NP")

print("\nTASK B: LIMITATIONS OF BASIC CFG")
print("""
1. Basic CFG does not represent semantic roles directly.
2. Relative clauses create nested structures.
3. Passive and active constructions can be difficult to interpret.
4. Multiple prepositional phrases create attachment ambiguity.
5. Long legal sentences can create many parse trees.
6. CFG does not naturally represent grammatical features.
7. CFG alone cannot determine who performed an action.
""")

print("TASK C: IMPROVED NLP ARCHITECTURE")
print("""
Legal Contract
     |
     v
Text Preprocessing
     |
     v
CFG Parsing
     |
     v
Feature Structures
     |
     v
Earley Parsing
     |
     v
PCFG Parse Ranking
     |
     v
Semantic Role Labeling
     |
     v
Legal Entity Extraction
     |
     v
Contract Information
""")

print("TASK D: STEP-BY-STEP INFORMATION EXTRACTION")

print("\nStep 1:")
print("Supplier -> Party involved")

print("\nStep 2:")
```

```
print("Delivered -> Action")

print("\nStep 3:")
print("Equipment -> Object")

print("\nStep 4:")
print("Company -> Recipient")

print("\nStep 5:")
print("Must replace defective components -> Obligation")

print("\nStep 6:")
print("Defective components -> Object of obligation")

print("\nStep 7:")
print("Within thirty days -> Deadline")

print("\nFinal Structured Output:")
print("Supplier = Supplier")
print("Action = Deliver")
print("Equipment = Equipment")
print("Company = Company")
print("Obligation = Replace defective components")
print("Deadline = Thirty days")

print("\nBenefits:")
print("""
PCFG improves ambiguity resolution.
Feature Structures enforce grammatical constraints.
Earley Parsing handles complex and nested sentences.
Semantic analysis identifies correct participants and actions.
The combined architecture improves contract analysis accuracy.
""")

print("=" * 65)
print("LEGAL CONTRACT ANALYSIS COMPLETED")
print("=" * 65)
```



```
print("=" * 65)
print("3. AI-BASED LEGAL CONTRACT ANALYSIS SYSTEM")
print("=" * 65)

sentence = "The supplier who delivered the equipment to the company must replace

print("\nTASK A: SYNTACTIC STRUCTURE")
print("\nContract Sentence:")
print(sentence)

print("\nMain Structure:")
print("The supplier must replace the defective components within thirty days.")

print("\nRelative Clause:")
print("who delivered the equipment to the company")

print("\nSemantic Roles:")
print("Supplier = Agent")
print("Delivered = Action")
print("Equipment = Object")
print("Company = Recipient")
print("Replace = Obligation")
print("Defective Components = Object")
print("Thirty Days = Deadline")

print("\nCFG Representation:")
print("S -> NP VP")
print("NP -> Det N RC")
print("RC -> RelPron VP")
print("VP -> V NP PP")
print("VP -> Modal VP")
print("VP -> V NP PP")
print("PP -> P NP")

print("\nTASK B: LIMITATIONS OF BASIC CFG")
print("""
```

```
Step 2:
Delivered -> Action

Step 3:
Equipment -> Object

Step 4:
Company -> Recipient

Step 5:
Must replace defective components -> Obligation

Step 6:
Defective components -> Object of obligation

Step 7:
Within thirty days -> Deadline

Final Structured Output:
Supplier = Supplier
Action = Deliver
Equipment = Equipment
Company = Company
Obligation = Replace defective components
Deadline = Thirty days

Benefits:

PCFG improves ambiguity resolution.
Feature Structures enforce grammatical constraints.
Earley Parsing handles complex and nested sentences.
Semantic analysis identifies correct participants and actions.
The combined architecture improves contract analysis accuracy.

================================================================
LEGAL CONTRACT ANALYSIS COMPLETED
```

4. AI-Based E-Commerce Product Search System

An e-commerce company is developing a conversational product-search system. Customers enter natural-language queries rather than selecting predefined filters.

The current system uses CFG but faces difficulties with:

- Ambiguous phrase attachment
- Coordination
- Missing words
- Informal customer queries
- Long product descriptions
- Real-time search requirements

Example user query:

"Find laptops with a touchscreen for students under ₹60,000."

The system may incorrectly associate **"for students"** with the touchscreen rather than the laptop.

Tasks:

a) Analyze the syntactic ambiguity in the given query and construct possible parse interpretations using CFG.

b) Evaluate the limitations of CFG in handling informal, elliptical, and ambiguous e-commerce search queries.

c) Propose an improved parsing architecture using **PCFG, Feature Structures, and Earley Parsing** to identify product attributes, user requirements, and price constraints.

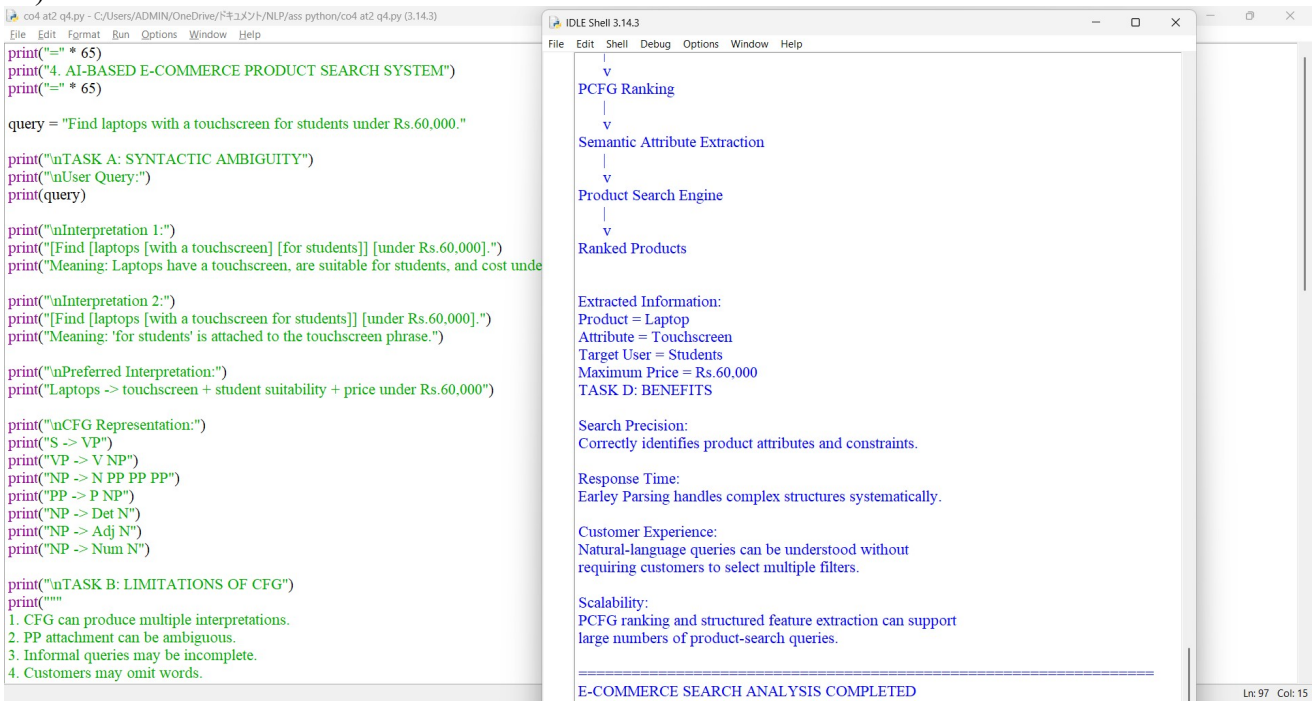
d) Justify how the proposed solution can improve search precision, response time, and customer experience in a large-scale e-commerce platform.

```
print("=" * 65)
print("4. AI-BASED E-COMMERCE PRODUCT SEARCH SYSTEM")
print("=" * 65)

query = "Find laptops with a touchscreen for students under Rs.60,000."

print("\nTASK A: SYNTACTIC AMBIGUITY")
```

```
print("\nUser Query:")
print(query)

print("\nInterpretation 1:")
print("[Find [laptops [with a touchscreen] [for students]] [under Rs.60,000].")
print("Meaning: Laptops have a touchscreen, are suitable for students, and cost under Rs.60,000.")

print("\nInterpretation 2:")
print("[Find [laptops [with a touchscreen for students]] [under Rs.60,000].")
print("Meaning: 'for students' is attached to the touchscreen phrase.")

print("\nPreferred Interpretation:")
print("Laptops -> touchscreen + student suitability + price under Rs.60,000")

print("\nCFG Representation:")
print("S -> VP")
print("VP -> V NP")
print("NP -> N PP PP PP")
print("PP -> P NP")
print("NP -> Det N")
print("NP -> Adj N")
print("NP -> Num N")

print("\nTASK B: LIMITATIONS OF CFG")
print("""
1. CFG can produce multiple interpretations.
2. PP attachment can be ambiguous.
3. Informal queries may be incomplete.
4. Customers may omit words.
5. Long product descriptions increase parsing complexity.
6. Coordination can create multiple structures.
7. CFG does not directly understand product meaning.
8. Basic CFG may not be fast enough for large-scale search.
""")

print("TASK C: IMPROVED PARSING ARCHITECTURE")
print("""
Customer Query
     |
     v
Query Preprocessing
     |
     v
CFG Parsing
     |
     v
Feature Structures
     |
     v
Earley Parsing
```

```
    |
    v
PCFG Ranking
    |
    v
Semantic Attribute Extraction
    |
    v
Product Search Engine
    |
    v
Ranked Products
""")

print("\nExtracted Information:")
print("Product = Laptop")
print("Attribute = Touchscreen")
print("Target User = Students")
print("Maximum Price = Rs.60,000")

print("TASK D: BENEFITS")
print("""
Search Precision:
Correctly identifies product attributes and constraints.

Response Time:
Earley Parsing handles complex structures systematically.

Customer Experience:
Natural-language queries can be understood without
requiring customers to select multiple filters.

Scalability:
PCFG ranking and structured feature extraction can support
large numbers of product-search queries.
""")

print("=" * 65)
print("E-COMMERCE SEARCH ANALYSIS COMPLETED")
```

```
print("="                                                                    *
65)
```



```
print("=" * 65)
print("4. AI-BASED E-COMMERCE PRODUCT SEARCH SYSTEM")
print("=" * 65)

query = "Find laptops with a touchscreen for students under Rs.60,000."

print("\nTASK A: SYNTACTIC AMBIGUITY")
print("\nUser Query:")
print(query)

print("\nInterpretation 1:")
print("[Find [laptops [with a touchscreen] [for students]] [under Rs.60,000].")
print("Meaning: Laptops have a touchscreen, are suitable for students, and cost unde

print("\nInterpretation 2:")
print("[Find [laptops [with a touchscreen for students]] [under Rs.60,000].")
print("Meaning: 'for students' is attached to the touchscreen phrase.")

print("\nPreferred Interpretation:")
print("Laptops -> touchscreen + student suitability + price under Rs.60,000")

print("\nCFG Representation:")
print("S -> VP")
print("VP -> V NP")
print("NP -> N PP PP PP")
print("PP -> P NP")
print("NP -> Det N")
print("NP -> Adj N")
print("NP -> Num N")

print("\nTASK B: LIMITATIONS OF CFG")
print("""
1. CFG can produce multiple interpretations.
2. PP attachment can be ambiguous.
3. Informal queries may be incomplete.
4. Customers may omit words.
```

```
    |
    v
PCFG Ranking
    |
    v
Semantic Attribute Extraction
    |
    v
Product Search Engine
    |
    v
Ranked Products


Extracted Information:
Product = Laptop
Attribute = Touchscreen
Target User = Students
Maximum Price = Rs.60,000
TASK D: BENEFITS

Search Precision:
Correctly identifies product attributes and constraints.

Response Time:
Earley Parsing handles complex structures systematically.

Customer Experience:
Natural-language queries can be understood without
requiring customers to select multiple filters.

Scalability:
PCFG ranking and structured feature extraction can support
large numbers of product-search queries.

================================================================
E-COMMERCE SEARCH ANALYSIS COMPLETED
```

5. Intelligent Airline Voice Assistant

An airline is developing a real-time voice assistant that allows passengers to modify flight bookings using spoken commands.

The system currently uses a conventional top-down parser. It experiences:

- Backtracking
- Ambiguous attachment
- Speech recognition errors
- Incomplete commands
- Delayed responses
- Difficulty interpreting corrections made by users

Example spoken command:

"Change my flight to Mumbai tomorrow with two passengers."

Depending on the parse, **"with two passengers"** may be incorrectly interpreted as an attribute of the destination rather than the number of passengers.

The passenger may also say:

"Change my flight to Mumbai tomorrow... actually, make that Bangalore."

Tasks:

a) Analyze the possible syntactic structures and ambiguities in the given voice command.

b) Evaluate the limitations of top-down parsing when processing incomplete, corrected, and ambiguous spoken commands in real time.

c) Design a parsing architecture using **Earley Parsing, PCFG, and Feature Structures** to support partial input, ambiguity resolution, and agreement constraints.

d) Develop a processing workflow showing how speech input can be converted into a structured booking request and justify the proposed method for real-time airline applications.

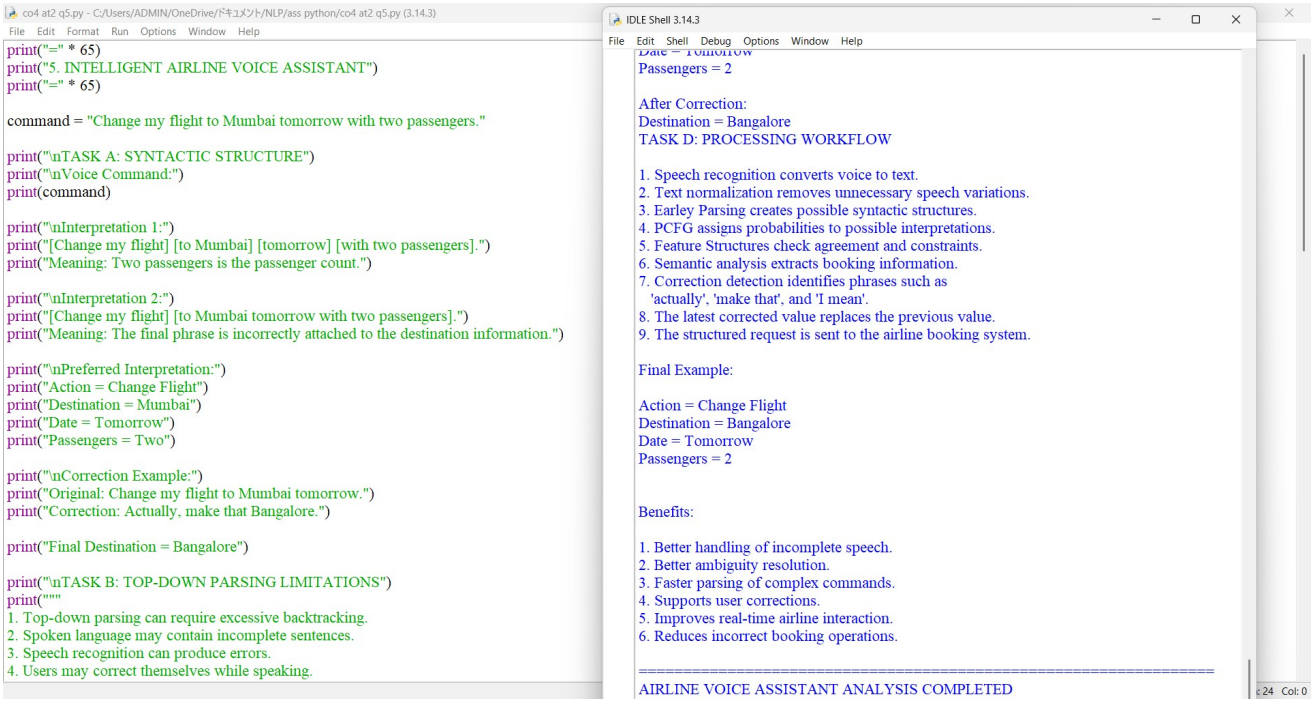
```
print("=" * 65)
print("5. INTELLIGENT AIRLINE VOICE ASSISTANT")
print("=" * 65)
```

```
command = "Change my flight to Mumbai tomorrow with two passengers."

print("\nTASK A: SYNTACTIC STRUCTURE")
print("\nVoice Command:")
print(command)

print("\nInterpretation 1:")
print("[Change my flight] [to Mumbai] [tomorrow] [with two passengers].")
print("Meaning: Two passengers is the passenger count.")

print("\nInterpretation 2:")
print("[Change my flight] [to Mumbai tomorrow with two passengers].")
print("Meaning: The final phrase is incorrectly attached to the destination information.")

print("\nPreferred Interpretation:")
print("Action = Change Flight")
print("Destination = Mumbai")
print("Date = Tomorrow")
print("Passengers = Two")

print("\nCorrection Example:")
print("Original: Change my flight to Mumbai tomorrow.")
print("Correction: Actually, make that Bangalore.")

print("Final Destination = Bangalore")

print("\nTASK B: TOP-DOWN PARSING LIMITATIONS")
print("""
1. Top-down parsing can require excessive backtracking.
2. Spoken language may contain incomplete sentences.
3. Speech recognition can produce errors.
4. Users may correct themselves while speaking.
5. Ambiguous phrase attachment can create incorrect structures.
6. Complex commands can increase response time.
7. Conventional parsing may not handle real-time corrections well.
""")

print("TASK C: IMPROVED PARSING ARCHITECTURE")
print("""
Speech Input
     |
     v
Speech Recognition
     |
     v
Text Normalization
     |
     v
Earley Parser
     |
     v
PCFG Ranking
```

```
    |
    v
Feature Structures
    |
    v
Semantic Role Extraction
    |
    v
Correction Detection
    |
    v
Structured Booking Request
    |
    v
Booking System
""")

print("\nFeature Structure:")
print("Action = Change")
print("Flight = Current Flight")
print("Destination = Mumbai")
print("Date = Tomorrow")
print("Passengers = 2")

print("\nAfter Correction:")
print("Destination = Bangalore")

print("TASK D: PROCESSING WORKFLOW")
print("""
1. Speech recognition converts voice to text.
2. Text normalization removes unnecessary speech variations.
3. Earley Parsing creates possible syntactic structures.
4. PCFG assigns probabilities to possible interpretations.
5. Feature Structures check agreement and constraints.
6. Semantic analysis extracts booking information.
7. Correction detection identifies phrases such as
  'actually', 'make that', and 'I mean'.
8. The latest corrected value replaces the previous value.
9. The structured request is sent to the airline booking system.

Final Example:

Action = Change Flight
Destination = Bangalore
Date = Tomorrow
Passengers = 2
""")

print("\nBenefits:")
print("""
1. Better handling of incomplete speech.
2. Better ambiguity resolution.
3. Faster parsing of complex commands.
```

4. Supports user corrections.
5. Improves real-time airline interaction.
6. Reduces incorrect booking operations.""")

print("=" * 65)
print("AIRLINE VOICE ASSISTANT ANALYSIS COMPLETED")
print("=" * 65)



Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution

		industry standards			
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Q. No. / Criteria Level	Understanding of Industry Problem	Application of Engineering Concepts	Solution Design	Analysis	Professionalism	Sub. Total	Total %
1	4	4	4	4	4	20	93
2	4	4	4	4	4	20	
3	4	4	4	3	3	18	
4	4	4	4	3	3	18	
5	4	4	3	3	3	17	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____